

Gestão Financeira PF + PJ, num só lugar

Um sistema para controlar contas, boletos, obrigações fiscais, cartões e investimentos de pessoa física e empresa — com visão separada e visão consolidada, acessível pelo navegador e instalável no Android.

Contas bancárias

Boletos

DARF/DAS

Cartões

Simulador de dívida

Web + App Expo

VERSÃO

1.3

DATA

**06 de agosto de
2026**

AUTOR

Deyvid

STATUS

Para revisão

Sumário

01	Sumário executivo	a tese e a recomendação de sequência
02	Problema, visão e princípios	o que o produto é e o que ele não é
03	Modos de uso e jornada	PF, PJ e visão consolidada
04	Arquitetura funcional	mapa de módulos e fases
05	Contas, saldos e extratos bancários	o que dá para automatizar hoje
06	Boletos por CPF e CNPJ	o módulo mais frágil da ideia original
07	Obrigações fiscais — DARF e DAS	por que isso não se automatiza sozinho
08	Cartões de crédito e investimentos	a mesma via das contas, com um cuidado a mais
09	Simulador de novo compromisso	o módulo de maior valor e menor risco
10	Segurança, LGPD e confiabilidade	o que muda quando o dado é financeiro
11	Stack técnica e hospedagem	React, Laravel, MySQL e os limites do Hostgator
12	Modelo de dados	entidades principais e cuidados estruturais
13	Roadmap em fases	F0 a F4, cada uma valendo sozinha
14	Decisões em aberto	o que só você pode responder
15	Riscos, métricas e próximos passos	o que fazer nesta semana

Sumário executivo

O produto em uma página: o que ele resolve, o que sustenta essa escolha, e em que ordem construir.

A ideia central é boa e resolve um problema real: hoje sua vida financeira está espalhada entre banco PF, banco PJ, boletos, cartões, obrigações fiscais e investimentos, sem um lugar único que mostre o todo e permita decidir "posso assumir essa dívida ou não". Um sistema assim tem valor mesmo que nenhuma integração externa funcione — porque o núcleo do valor é a **visão consolidada** e o **simulador de compromisso**, não a automação de coleta de dados.

TESE CENTRAL

As três integrações mais desejadas — boletos por CPF/CNPJ, extratos bancários automáticos e cartões de crédito — têm níveis de viabilidade muito diferentes entre si hoje. Uma é inviável para uma pessoa física construir sozinha, duas são viáveis através de um agregador de Open Finance (não direto do Banco Central). Tratar as três como "a mesma integração, fases diferentes" é o erro que mais atrasaria este projeto. O caminho certo é separar o que **já funciona com cadastro manual** (boletos, DARF/DAS) do que **pode ser automatizado via terceiro regulado** (contas, extratos, cartões) e lançar o valor manual primeiro.

1.1 O que este documento decide

Este documento valida a ideia, aponta onde ela precisa de ajuste antes de virar código, e propõe uma sequência de construção em fases — cada uma entregando algo usável sozinho. Ele não é uma especificação técnica linha a linha; é o mapa que orienta a especificação técnica seguinte.

HISTÓRICO DE REVISÕES

v1.1: seis decisões fechadas (Laravel 13, multiempresa, agregador Pluggy, autenticação, veredito sobre boletos) e seis enriquecimentos de escopo a partir de uma revisão externa: contas a pagar **e a receber**, categorias, fluxo de caixa projetado, metas financeiras, dashboards detalhados e uma futura fase F4. Um ponto da revisão externa — seção genérica de "público-alvo" — foi deliberadamente **não incorporado** (razão em 2.6).

v1.2: a stack (capítulo 11) passa a adotar como base literal o repositório pessoal `laravel-base` e o padrão "Padrões Laravel — DMTA" que você já usa em todo projeto PHP novo — confirma Laravel 13/PHP 8.3, formaliza a arquitetura em UseCases/Services/Domain/Models, e adiciona um ajuste de segurança que ainda não estava explícito (document root fora de `public_html`).

v1.3: sete decisões fechadas de uma vez (D-04, D-06, D-09, D-11 a D-14) — o produto continua de uso pessoal (D-11); simulador ganha gráficos, CET e cenários (D-04); motor de e-mail e bot do Telegram viram porta de entrada prioritária (D-06); o app deixa de ser PWA/TWA e passa a ser um app Expo nativo consumindo API versionada (D-09); categorias ganham subcategorias customizáveis (D-12); metas entram na F1 (D-13); investimentos seguem manuais, sem rentabilidade automática (D-14). Só **D-07** (Integra Contador) segue em aberto, agora com o custo e a mecânica explicados (capítulo 07).

FASE	ENTREGA	DEPENDE DE	VIABILIDADE
F0	Cadastro manual de contas (a pagar e a receber), boletos, cartões e DARF/DAS (PF+N empresas), com categorias e subcategorias; dashboards separados e consolidado	Só desenvolvimento	Alta
F1	Simulador de compromisso (gráficos, CET, cenários) + fluxo de caixa projetado + motor de e-mail + bot do Telegram + metas financeiras	Dados da F0	Alta
F2	Extratos e saldos automáticos via Open Finance (Pluggy)	Cadastro na Pluggy	Média
F3	App Expo (Android/iOS) consumindo API versionada + cartões via Open Finance	F2	Média

NÃO FAÇA ISSO

Não trave o lançamento esperando a "consulta automática de boletos direto do Banco Central" funcionar. Ela não existe como serviço público de consulta (capítulo 06 explica o porquê). Se essa peça virar bloqueio de cronograma, o projeto nunca sai do papel — e é exatamente a peça menos essencial: cadastrar um boleto manualmente leva 20 segundos.

1.2 Recomendação de sequência

Comece pelo que não depende de ninguém além de você: modelagem de dados, cadastro manual e o simulador. Isso já resolve 70% da dor descrita ("não tenho visão geral, não sei se posso assumir uma dívida nova"). Só depois disso funcionando é que vale investir tempo e, possivelmente, dinheiro em integrações com terceiros regulados.

Problema, visão e princípios

Antes de desenhar tela, alinhar o que o produto resolve — e o que ele deliberadamente não tenta resolver.

2.1 O problema

DOR HOJE	CONSEQUÊNCIA
Contas PF e PJ em bancos diferentes, sem visão única	Decisão financeira tomada com informação parcial
Boletos, DARF e DAS espalhados em e-mail, apps de banco e portal da Receita	Risco de atraso e multa por esquecimento
Cartões de crédito com faturas em datas e apps diferentes	Dificuldade de saber o comprometimento real da renda
Nenhum lugar para simular "cabe no orçamento assumir essa parcela nova?"	Decisão de crédito no impulso, sem base numérica

2.2 Visão

Um painel único, com dono único, que mostra a saúde financeira de pessoa física e empresa lado a lado — nunca misturadas — e que responde, em segundos, se um novo compromisso financeiro cabe no orçamento antes de você assumi-lo. Em uma frase: **centralizar toda a informação financeira num único lugar para decidir melhor**, mesmo enquanto parte do dado ainda é cadastrada à mão.

2.3 Princípios

Separação estrita PF/PJ

Cada contexto tem saldo, extrato e obrigações próprios. Consolidar é *somar para enxergar*, nunca *misturar para movimentar*.

Manual primeiro, automático depois

Toda automação de dados externos precisa de um caminho manual equivalente, que funciona no dia 1 e continua funcionando se a integração cair.

Decisão, não só registro

O produto não é uma planilha bonita. Ele existe para responder "posso assumir isso?" — o simulador é tão importante quanto o cadastro.

Dado sensível, tratamento sensível

Token de banco, saldo, CPF/CNPJ: tudo isso pede criptografia, MFA e trilha de auditoria desde a primeira versão, não como "melhoria futura".

Um backend, contratos estáveis

Web (React) e app (Expo) são bases de código separadas, mas falam com a mesma API versionada e a mesma regra de negócio (capítulo 11) — a duplicação fica só na tela, nunca na lógica.

Honestidade sobre o automático

Se um dado veio de importação manual, a tela diz isso. Nunca finja uma automação que não existe.

2.4 O que este produto NÃO é

FORA DE ESCOPO — DE PROPÓSITO

Não é um sistema contábil substituto de contador — ele não gera SPED, não fecha balanço, não substitui a escrituração fiscal do CNPJ.

Não é um agregador que promete "conectar todos os bancos do Brasil" desde o dia 1 — começa pelas suas contas específicas.

Não é uma corretora ou gestor de investimentos — ele *mostra* a posição de investimentos, não opera nem recomenda ativos.

Não é multiusuário/colaborativo na v1 — é uma ferramenta de uso pessoal, o que simplifica muito a segurança.

2.5 Princípios de interface

Até 3 cliques para o essencial

Saldo, próximo vencimento e "posso assumir isso?" precisam estar a no máximo três toques de distância do login.

Android em primeiro lugar

Toda tela é pensada primeiro para o celular, depois adaptada para o desktop — é onde você provavelmente vai consultar o app no dia a dia.

Origem e frescor sempre visíveis

Generaliza o que os capítulos 06 e 09 já pedem: todo número mostra de onde veio o dado e há quanto tempo foi atualizado.

2.6 Sobre "para quem é isso" — por que não há uma seção de público-alvo

DECISÃO FECHADA — D-11

Uma revisão externa deste documento sugeriu adicionar uma seção de "público-alvo" (pessoas físicas, MEIs, autônomos, pequenos empresários). Não incorporei — e o motivo importa mais que a decisão em si. Escrever "público-alvo: pequenos empresários e autônomos" pressupõe um produto que outras pessoas vão usar, o que muda cadastro, isolamento de dados e o modelo de segurança inteiro (capítulo 10). Você confirmou: o mercado já tem soluções consolidadas para isso, e não vale o esforço de transformar este projeto num produto agora. **O sistema continua de uso pessoal** — o princípio 2.4 permanece valendo sem ressalva, e o modelo de segurança do capítulo 10 segue dimensionado para um usuário só.

Modos de uso e jornada

O sistema vive em múltiplos contextos: Pessoa Física, uma ou mais Pessoas Jurídicas, e a leitura consolidada de todos.

O usuário entra, escolhe (ou já vê, se preferir) um contexto — PF, PJ ou Consolidado — e todas as telas seguintes (contas, boletos, fiscal, cartões, simulador) respeitam esse filtro. Trocar de contexto é uma troca de lente sobre os mesmos módulos, não uma troca de aplicativo.

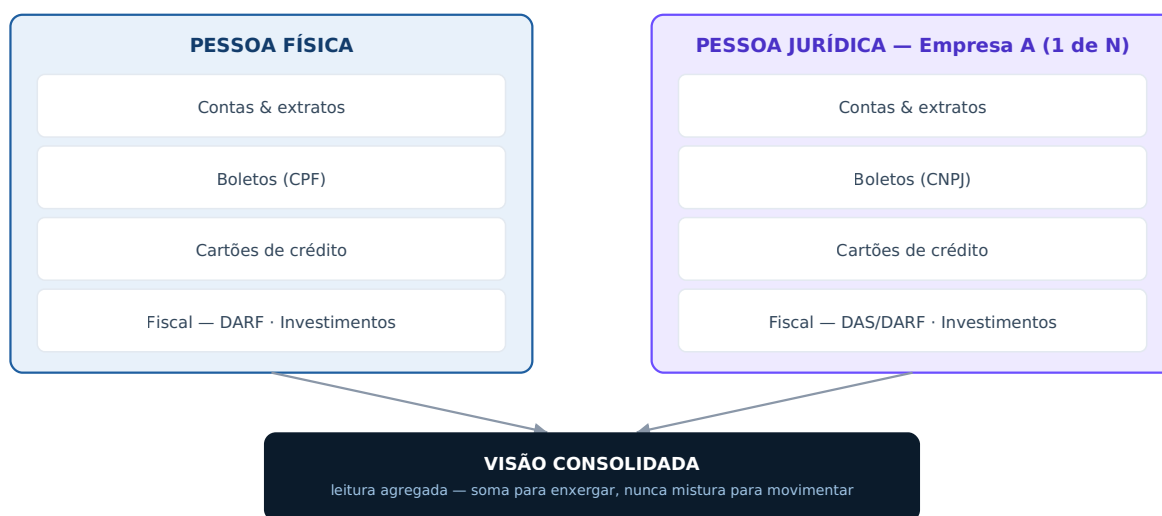


Figura 1 — Os contextos de uso. PF e cada empresa (Empresa A, B...) têm dados próprios; o consolidado soma todos, mas é leitura por cima, não fusão de saldos.

INSIGHT QUE MUDA O DESENHO — NÃO SÓ DE INTERFACE

Misturar saldo de PF e PJ num único "total disponível" não é apenas uma escolha de UX ruim — é um risco jurídico e contábil real. No direito brasileiro, tratar patrimônio de pessoa física e de empresa como se fossem a mesma coisa é chamado de **confusão patrimonial** e é um dos motivos que justificam a desconsideração da personalidade jurídica (o "véu" que protege seu patrimônio pessoal em caso de dívida da empresa). O consolidado deste sistema deve **somar para exibir** ("seu patrimônio líquido total, olhando os dois lados"), mas nunca sugerir que dinheiro de um lado pode cobrir o outro sem registrar isso como o que de fato é: um empréstimo entre você e sua empresa, ou uma distribuição de lucros/pró-labore.

3.1 Jornada típica

1. **Entrar** — login com MFA, escolher contexto padrão (o sistema lembra a última escolha).
2. **Ver o dashboard do contexto** — saldo agregado das contas, próximos vencimentos (boletos, DARF/DAS, faturas de cartão), alerta se algo está atrasado.
3. **Registrar ou revisar** — cadastrar um boleto novo, confirmar uma DAS do mês, revisar uma fatura importada.

4. **Simular antes de decidir** — antes de assumir uma parcela nova, rodar o simulador (capítulo 09) usando o orçamento real do contexto certo.
5. **Olhar o consolidado** — de tempos em tempos, ver a fotografia de todos os contextos juntos, sem que isso vire operação.

3.2 Multiempresa — um usuário, N empresas D-03 decidida

Hoje você tem uma empresa; amanhã pode ter mais. A boa notícia é que isso não muda a arquitetura, só a cardinalidade: o conceito certo desde o início era `Context`, e um `Context` de PJ simplesmente aponta para uma `Company` (capítulo 12). Cadastrar uma segunda empresa é criar uma nova `Company` e um novo `Context` — o seletor de contexto no topo da tela passa a listar "Pessoa Física", "Empresa A", "Empresa B" (e assim por diante), cada uma com suas próprias contas, boletos, cartões e obrigações fiscais.

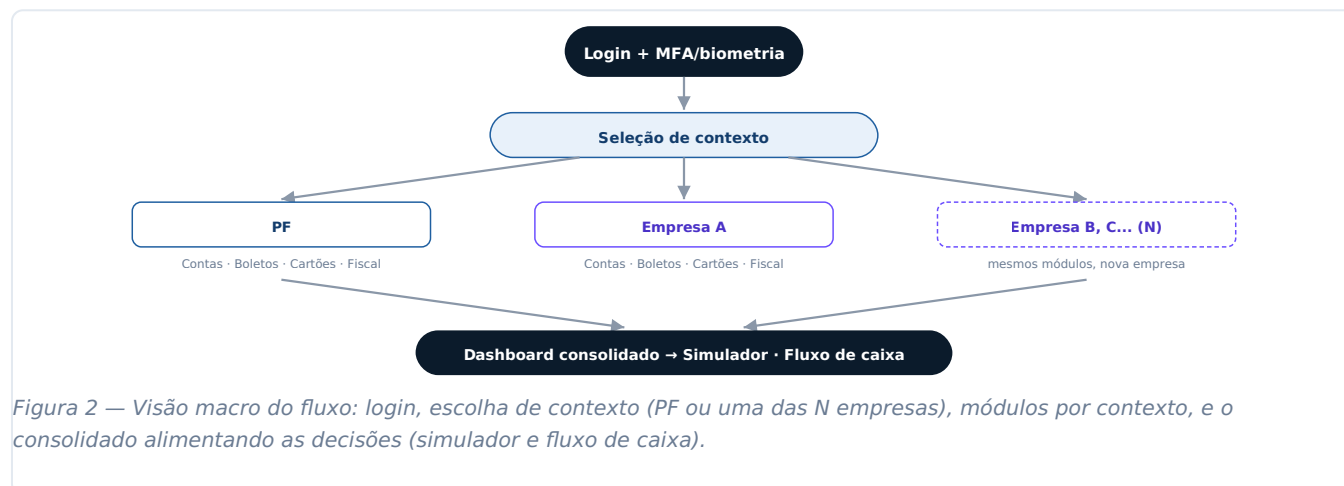
POR QUE ISSO É BARATO AGORA E CARO DEPOIS

Modelar para "no máximo uma PJ" e depois precisar de duas é o tipo de mudança que obriga a alterar chave estrangeira em quase toda tabela do sistema. Modelar para N empresas desde a F0 custa quase nada a mais — é a mesma tabela `Context`, só sem a suposição de unicidade. O consolidado passa a somar **todos** os contextos (PF + todas as empresas), mantendo a mesma regra do capítulo 03: soma para exibir, nunca mistura para movimentar entre empresas diferentes.

Arquitetura funcional

Oito módulos apoiados num núcleo compartilhado. A etiqueta de cada módulo mostra em que fase ele nasce.

4.1 Visão macro do fluxo



4.2 Mapa de módulos



Note que quatro módulos (contas, boletos, fiscal, cartões) seguem o mesmo padrão: **nascem manuais e ganham automação depois**. Isso não é preguiça de design — é a única forma sensata de lançar rápido um produto que depende de três integrações externas com graus de maturidade diferentes (capítulos 05 a 08 detalham cada uma).

Dois ajustes de escopo entraram nesta revisão, os dois dentro do módulo "Contas & Extratos" — sem exigir módulo novo: primeiro, ele passa a cobrir **contas a receber** (salário, pró-labore, clientes, aluguel

recebido, Pix esperado), não só a pagar — sem o lado da entrada, o fluxo de caixa do capítulo 09 fica cego pela metade. Segundo, todo lançamento (conta, boleto, fatura de cartão) ganha uma **categoria** (ex.: Alimentação, Impostos, Salário, Marketing) — é o que sustenta relatórios, orçamento por categoria e o cálculo de "despesas fixas" que já alimenta o orçamento livre do simulador.

4.3 Dashboards por contexto

Cada contexto (PF, cada empresa, e o consolidado) tem seu próprio dashboard — não é a mesma tela com números trocados, é um recorte do que importa em cada nível.

Dashboard PF

Saldo total, contas, cartões, boletos a pagar e a receber, investimentos, despesas e receitas do mês, próximos vencimentos.

Dashboard por empresa

Caixa, impostos (DAS/DARF), contas a pagar e a receber, faturamento do mês, despesas por categoria.

Dashboard consolidado

Patrimônio líquido total, aberto por PF e por empresa, evolução patrimonial no tempo, e os compromissos futuros de todos os contextos juntos.

ANTIPADRÃO A EVITAR

Não fature o dashboard consolidado como "seu saldo disponível para gastar" — ele é diagnóstico (capítulo 03), e um número de patrimônio total ao lado de um botão de simular gasto é um convite implícito a tratar dinheiro de empresa como se fosse pessoal. Mostre o total, mas mantenha a ação de simular sempre dentro de um contexto específico.

4.4 Camada de captura multiplataforma D-06 decidida

Todo lançamento — boleto, despesa avulsa, receita — pode entrar no sistema por quatro portas diferentes. As quatro terminam no mesmo caso de uso (`RegisterBill` , `RegisterTransaction`); a única coisa que muda é de onde veio o dado, guardado no campo `origem` (capítulo 02, princípio de honestidade sobre o automático).

Motor de e-mail

Você encaminha ou copia um e-mail com boleto para um endereço próprio do sistema. O backend extrai o PDF anexo, lê a linha digitável/código de barras, pré-preenche o cadastro como pendência de confirmação.

Scanner de código de barras

No app (capítulo 11), a câmera lê a linha digitável de um boleto físico ou de tela — decodifica valor, vencimento e beneficiário sem digitar nada.

Bot do Telegram

Lançamento rápido de despesas, receitas e boletos por mensagem — sem abrir o app. Ex.: "gastei 45 no mercado" vira um lançamento pendente de categorização.

Cadastro manual

Sempre disponível, web ou app — é o piso que garante que o sistema funciona mesmo se as outras três portas falharem.

Os detalhes técnicos de cada porta — como o motor de e-mail lê o anexo, como o bot do Telegram interpreta uma mensagem — estão nos capítulos 06 (boletos) e 11 (stack).

Contas, saldos e extratos bancários

Viável — mas não do jeito que a ideia original supõe. Não existe integração direta e gratuita "banco a banco"; existe um intermediário regulado no meio.

5.1 O que não é viável construir sozinho

Não existe uma API pública do Banco Central que devolva saldo e extrato de qualquer conta a partir do seu CPF/CNPJ. O que existe é o **Open Finance Brasil**: um padrão regulado pelo BC em que instituições financeiras compartilham dados *mediante seu consentimento explícito*, mas apenas com participantes credenciados no ecossistema. Se você tentasse implementar isso "na unha", precisaria se tornar uma instituição participante do Open Finance — processo caro, demorado e desproporcional para um projeto pessoal.

5.2 O que é viável: agregadores licenciados

Empresas como **Pluggy** e **Belvo** já são credenciadas no Open Finance Brasil (a Pluggy, por exemplo, é autorizada pelo Banco Central como Iniciadora de Transação de Pagamento) e vendem exatamente essa ponte: seu backend chama a API delas, o usuário autoriza o acesso ao banco pelo fluxo de consentimento, e elas devolvem contas, saldos e extratos já tratados. A Pluggy tem um plano de acesso pessoal gratuito ("Meu Pluggy") que serve bem para um projeto como este, ao menos na validação.

Decisão fechada (D-05): o projeto segue com Pluggy.

RECOMENDAÇÃO

Trate contas e extratos como um **fornecedor externo plugável**, não como parte do núcleo do sistema. Um serviço (`BankAggregatorService``, no padrão de casos de uso do capítulo 11) isola a chamada ao Pluggy/Belvo — se um dia trocar de fornecedor, ou se o fornecedor sair do ar, o resto do sistema não sente.

5.3 Caminho manual (sempre disponível)

Cadastro de conta com saldo informado à mão, e importação de extrato via arquivo (OFX é o formato padrão que praticamente todo internet banking exporta; CSV como alternativa). Isso garante que o produto funciona mesmo sem nenhuma integração — e é o que entra na F0.

Boletos por CPF e CNPJ

Este é o módulo onde a ideia original precisa mudar de forma. Não existe hoje uma consulta pública "todos os boletos emitidos no meu nome".

O QUE A PESQUISA MOSTROU

SIGEF (sistema do Banco Central) só confirma o status de *um* boleto específico, e exige que você já tenha o código identificador de 10 dígitos do próprio boleto — ele não lista boletos novos, você tem que já saber que eles existem.

Registrato (também do BC) é um portal onde você solicita relatórios sobre contas, empréstimos e chaves Pix vinculados ao seu CPF/CNPJ — não inclui boletos, exige login gov.br, e entrega um relatório por pedido manual, não uma API.

DDA (Débito Direto Autorizado) é o mecanismo real que mostra "todos os boletos emitidos no seu nome" — mas ele vive *dentro do internet banking do seu banco*, é uma adesão bancária, não uma API pública que qualquer aplicação possa consultar livremente.

Conclusão honesta: hoje, para uma pessoa física construir isso sozinha, **não existe atalho**. A alternativa realista é cadastro manual — e nisso dá para ser inteligente.

6.1 Se todo boleto é registrado, por que não dá para consultar?

A pergunta certa: os boletos são centralizados — desde a criação do DDA pela Febraban em 2009, os bancos são obrigados a registrar todo boleto emitido numa base única mantida pela CIP (Câmara Interbancária de Pagamentos). Por isso qualquer banco consegue mostrar "todos os boletos emitidos no seu CPF/CNPJ" dentro do próprio internet banking, não importa qual banco emitiu. O problema não é a inexistência do registro central; é que o **acesso a ele é restrito a instituições financeiras autorizadas** (bancos, participantes do DDA), não aberto a qualquer aplicação de terceiro. É a mesma lógica do Open Finance (capítulo 05): o dado existe e é centralizado, mas só quem é credenciado como participante do sistema financeiro consulta diretamente — um app pessoal não vira "banco" só para ler essa informação.

Isso reforça, e não enfraquece, a decisão **D-08: sem automação** por enquanto. O único caminho realista de automação futura seria via um agregador de Open Finance que decida expor dado de DDA como produto (nenhum dos que pesquisamos — Pluggy, Belvo — anuncia isso hoje) — o que vira uma reavaliação de fase avançada, não um bloqueio para começar.

6.2 Quatro formas de enriquecer o cadastro manual

Motor de e-mail — prioridade 1

Um endereço próprio do sistema (ex.: `boletos@seudominio.com.br`) recebe o e-mail com o boleto anexado (encaminhado por você, ou direto do emissor com regra de encaminhamento automático do seu provedor). O backend extrai o PDF, lê a linha digitável/código de barras e preenche o cadastro como pendência de confirmação.

Leitura de código de barras

No app (capítulo 11), abrir a câmera e escanear a linha digitável/código de barras do boleto físico ou de tela — decodifica valor, vencimento e beneficiário sem digitar nada.

Bot do Telegram — prioridade 1

Além de boletos, serve para lançar qualquer despesa ou receita avulsa por mensagem de texto — o canal mais rápido do sistema, porque não exige abrir nada.

DDA por adesão bancária (avançado)

Alguns bancos expõem parte dos dados de DDA via Open Finance. Vale investigar caso a caso, mas não é recurso padronizado — trate como experimento de fase distante, não como base do módulo.

DECISÃO FECHADA — D-08 / D-06

Sem consulta automática do BC (D-08) — cadastro manual continua sendo a via definitiva, mas "manual" aqui significa **enriquecido por automação própria**: motor de e-mail e bot do Telegram são as duas portas de entrada prioritárias (D-06), por não dependerem de nenhum agregador nem de limite de contas/instituições — você não precisa de uma "quantidade" fixa de integrações porque essas duas portas capturam qualquer boleto ou lançamento, de qualquer fonte, contanto que você encaminhe o e-mail ou mande a mensagem.

6.3 Como o motor de e-mail funciona, tecnicamente

Ler um boleto anexado exige duas coisas que a maioria dos serviços de e-mail transacional (como o MailerSend do padrão DMTA, capítulo 11) não faz: receber e-mail de *entrada*, não só enviar. A solução prática é um provedor com parse de e-mail de entrada (ex.: Mailgun, SendGrid) que recebe em `boletos@seudominio.com.br` e chama um webhook do Laravel com o PDF anexo — ou, mais simples e sem depender de mais um fornecedor, uma caixa IMAP dedicada consultada pelo mesmo cron que já roda `schedule:run` (capítulo 11), baixando anexos novos a cada execução. A leitura do código de barras dentro do PDF usa uma biblioteca de leitura de barcode (ex.: baseada em ZXing) sobre a imagem extraída da página.

6.4 Como o bot do Telegram funciona, tecnicamente

O Telegram Bot API é gratuito, bem documentado e não exige processo de revisão de app store — você cria o bot em minutos pelo @BotFather e recebe mensagens via webhook (o mesmo Laravel que já serve a API recebe o POST do Telegram). Não é exatamente "open source" no sentido de você hospedar seu próprio servidor — é um serviço da Telegram —, mas o que importa aqui é o mesmo: API estável, gratuita, sem OAuth complicado, com pacotes prontos para Laravel (ex.: `irazasyed/telegram-bot-sdk`).

Uma conversa guiada simples (o bot pergunta valor, categoria, contexto quando faltar informação) evita precisar de processamento de linguagem natural para a v1.

Obrigações fiscais — DARF e DAS

Mesma lógica dos boletos: sem automação de consulta viável para pessoa física comum. A saída aqui é ainda mais simples, porque os vencimentos são previsíveis.

7.1 Por que não dá para automatizar direto

A Receita Federal disponibiliza o **Integra Contador**, uma API que automatiza emissão e consulta de DARF, DAS e outros documentos — mas ela exige certificado digital e-CNPJ e é desenhada para contadores e escritórios com procuração eletrônica, não para consumo pessoal direto de um app. Usar e-CAC via scraping (simular login e "ler a tela") não é uma opção confiável: quebra a cada mudança de layout do governo e é a receita clássica para um sistema frágil.

7.1.1 Por que o e-CNPJ, e quanto custa

O e-CNPJ não é uma exigência arbitrária: o Integra Contador emite guias oficiais e consulta dado protegido de contribuinte em nome da sua empresa, então a Receita exige uma identidade digital forte (padrão ICP-Brasil) para autenticar isso — é a mesma lógica de segurança do capítulo 10, aplicada pelo governo. Respondendo direto:

PERGUNTA	RESPOSTA
É gratuito?	Não, em dois níveis. O certificado e-CNPJ custa entre R\$ 150 e R\$ 280/ano (modelo A1, pesquisa de mercado 2026 — varia por certificadora). A contratação do Integra Contador em si é gratuita, mas o uso é cobrado por consumo (Loja Serpro, pós-pago por chamada de API, com desconto por volume) — para o volume baixo de um usuário pessoal, o custo mensal tende a ser pequeno, mas não é zero.
É seguro?	Sim — é o canal oficial da Receita Federal/Serpro, o mesmo que contadores usam.
Recalcula automaticamente?	Sim. O SICALC (para DARF) e o PGDAS-D (para DAS/Simples Nacional) — ambos acessados pelo Integra Contador — recalculam juros e multa por atraso automaticamente com base na data de pagamento informada. É a principal vantagem sobre o motor de obrigações recorrentes deste documento, que não faz esse recálculo sozinho.

VALE A PENA AGORA?

Para o cenário de hoje — pagar em dia, sem recalculando atraso com frequência — o motor de obrigações recorrentes (7.2) cobre o essencial sem custo recorrente. O Integra Contador começa a valer a pena se: você atrasa pagamentos com alguma frequência (o recálculo automático evita erro de conta), ou se decide contratar contabilidade formal com e-CNPJ por outro motivo — nesse caso a integração vira quase gratuita, porque o certificado já existiria de qualquer forma. **D-07 continua aberta**: revise depois de rodar a F0/F1 por alguns meses e ver se atraso é um problema real.

7.2 O que fazer em vez disso

Os vencimentos de DAS (Simples Nacional) e da maioria dos DARFs são previsíveis por regra — DAS vence todo dia 20; IRPJ/CSLL trimestral tem calendário fixo; ganho de capital tem prazo de 30 dias da operação. Isso permite um recurso que vale mais do que parece:

IDEIA DE ENRIQUECIMENTO

Um **motor de obrigações recorrentes**: você cadastra a regra uma vez ("DAS mensal, PJ, vence dia 20"), e o sistema gera automaticamente o lançamento do mês seguinte como pendência a confirmar — junto com o valor do mês anterior como sugestão. Isso substitui 90% do valor de uma integração fiscal automática, sem depender de nenhuma API externa frágil.

Se no futuro você formalizar um contrato de contabilidade com acesso a e-CNPJ, aí sim o Integra Contador vira uma integração avançada plausível — mas isso é decisão de fase distante, não de v1 (ver decisão **D-07**, capítulo 14).

Cartões de crédito e investimentos

Cartões seguem exatamente o padrão do módulo de contas. Investimentos merecem um cuidado a mais: nem tudo que parece automatizável é barato de automatizar.

A Pluggy (fornecedor decidido em D-05) também traz dados de cartão de crédito (limite, fatura aberta, fatura fechada, parcelamentos) como parte do mesmo consentimento de Open Finance usado para contas — não é uma integração separada do ponto de vista de fornecedor, só um tipo de dado a mais dentro do mesmo `BankAggregatorService`.

NÃO FAÇA ISSO

Não espere a integração de cartão estar pronta para lançar o simulador (capítulo 09). Os quatro números que o simulador precisa de um cartão — limite total, limite disponível, dia de fechamento, dia de vencimento — cabem num cadastro manual de 30 segundos. Represar o módulo de maior valor do produto atrás do módulo de integração mais incerta é a forma mais comum de um projeto pessoal nunca ser lançado.

8.1 Dado mínimo do cadastro manual

CAMPO	USO NO SIMULADOR
Limite total	Teto de comprometimento por cartão
Limite disponível / fatura atual	Espaço restante antes de uma nova compra parcelada
Dia de fechamento	Em que fatura a próxima parcela cai
Dia de vencimento	Data-base para projetar o fluxo de caixa

8.2 Investimentos — até onde vale automatizar

Registrar **posição e aportes** manualmente (quanto você tem, quanto aportou por mês) já entrega a maior parte do valor: entra no patrimônio consolidado e no cálculo de reserva. O que fica caro é a **rentabilidade automática** — calcular o retorno real de uma carteira exige cotação de mercado atualizada (ações, fundos, renda fixa marcada a mercado), o que significa mais uma integração externa, agora com uma fonte de dados financeiros de mercado, não bancários.

DECISÃO FECHADA — D-14

Sem rentabilidade automática por ora. A mesma lógica de D-11 (capítulo 2.6) se aplica aqui: sem intenção de transformar isto num produto, não vale investir numa integração de dados de mercado (cotações, índices — tipo B3 Market Data, Brapi) só para um cálculo que dá para acompanhar manualmente. Posição e aportes manuais continuam sendo o suficiente; revisita-se essa decisão só se a carteira crescer a ponto de o cálculo manual virar fricção real.

Simulador de novo compromisso

O módulo de maior valor e menor risco técnico do projeto inteiro — não depende de nenhuma integração externa.

Você informa um valor e um número de parcelas; o sistema responde quanto isso consome do seu orçamento livre mensal, se cabe agora ou a partir de quando cabe, quanto de juros você vai pagar ao todo, e como fica a comparação antes/depois no contexto certo (PF ou PJ). **Decisão fechada (D-04):** esse é o escopo confirmado — as seções 9.3 a 9.6 detalham os enriquecimentos que entram junto.

Simulador de novo compromisso — Pessoa Jurídica

Nova parcela

Valor total
R\$ 18.500,00

Parcelas
12x

Categoria
Equipamento

Comprometimento do orçamento livre

35% do orçamento livre mensal do contexto PJ — acima do teto de 30% configurado

Baseado em: receita média (3 meses) — despesas fixas — dívidas já assumidas
Dados atualizados há 2 dias · qualidade: alta

Quando cabe com segurança

- ✗ Agora — ultrapassa o teto de 30%
- ✓ A partir de março/2027, após quitar 2 boletos recorrentes

Recalculado sempre que um dado do contexto PJ muda.

Simular

Figura 4 — Simulador: entrada simples (valor, parcelas), resposta em duas frentes (% do orçamento e "a partir de quando").

9.1 Como o "orçamento livre" é calculado

Orçamento livre do contexto = receita média recente — despesas fixas recorrentes (boletos, DAS, aluguel) — parcelas de cartão e dívidas já assumidas. O simulador aplica a parcela nova sobre esse número e compara contra um teto de comprometimento configurável (sugestão inicial: 30%, ajustável por você).

O SIMULADOR É TÃO BOM QUANTO OS DADOS QUE O ALIMENTAM

Se as contas, boletos e cartões do contexto não estiverem atualizados, o percentual mostrado é uma ilusão de precisão. Por isso a tela mostra "dados atualizados há N dias" — um simulador que parece preciso mas usa dado velho é pior do que nenhum simulador, porque gera falsa confiança. Nunca esconda essa informação de frescor do usuário.

9.2 Enriquecimento proposto: semáforo de comprometimento

Verde (abaixo do teto), amarelo (dentro de uma margem de tolerância, ex.: até 10% acima), vermelho (estourou o teto). Simples de implementar, e transforma um número abstrato numa decisão visual imediata — o tipo de recurso que vale mais que uma tela cheia de gráficos.

9.3 Fluxo de caixa futuro (7 / 30 / 90 dias)

O simulador responde "cabe uma dívida nova?". O fluxo de caixa responde a pergunta vizinha, igualmente importante: "vou ter saldo suficiente daqui a duas semanas?". Ele projeta entradas (contas a receber, salário, clientes) e saídas (boletos, DAS/DARF, faturas de cartão) já cadastradas, num horizonte curto — sem nenhuma integração nova, é o mesmo dado do capítulo 04 olhado no tempo em vez de olhado no total.

HORIZONTE	ENTRADAS PREVISTAS	SAÍDAS PREVISTAS	SALDO PROJETADO
Hoje	—	—	R\$ 8.400
+7 dias	Cliente A — R\$ 3.200	Fatura cartão — R\$ 2.100	R\$ 9.500
+30 dias	Salário/pró-labore — R\$ 6.000	DAS + 2 boletos — R\$ 4.300	R\$ 11.200
+90 dias	Projeção com base na média de 3 meses	Projeção com base na média de 3 meses	R\$ 14.800 estimado

POR QUE ISSO É BARATO E VALE A PENA

É pura leitura e soma sobre dados que já existem no modelo (capítulo 12) — sem chamada externa, sem risco novo. Por isso entra na F1 junto com o simulador, não numa fase separada: é o mesmo motor de orçamento, só que espalhado no tempo em vez de comprimido num número só.

9.4 Quanto de juros você vai pagar

Além da parcela, o simulador mostra o **custo total da operação** (soma de todas as parcelas menos o valor à vista) e o **CET** — Custo Efetivo Total, a taxa que já embute juros, tarifas e o efeito do prazo, mês a mês. É o número que de fato compara duas ofertas diferentes; taxa de juros nominal sozinha engana quando o número de parcelas muda.

9.5 Comparar cenários

Rodar duas simulações lado a lado — por exemplo "R\$ 18.500 em 12x" contra "R\$ 18.500 à vista com 8% de desconto", ou "12x no cartão" contra "6x direto com o fornecedor" — vendo as duas barras de comprometimento e os dois totais de juros na mesma tela. Sem isso, cada simulação vive isolada e "qual caminho é mais barato" volta a depender de conta de cabeça.

9.6 O mês mais apertado

Uma parcela de 12x não pesa igual em todos os meses — se o orçamento livre de fevereiro já está baixo (13º já gasto, IPVA), a mesma parcela pode ultrapassar o teto só naquele mês, mesmo que a média do ano feche bem. O simulador cruza os meses da parcela com o fluxo de caixa projetado (9.3) e aponta o **pior mês**, não só a média — é a diferença entre "cabe no papel" e "cabe de verdade".

9.7 Metas financeiras D-13 decidida

Um objetivo com valor-alvo, prazo e progresso — reserva de emergência, capital de giro, troca de equipamento. Cada meta soma automaticamente os aportes que você marcar como destinados a ela e mostra o percentual concluído. Entra na **F1**, junto com o simulador: não depende de nenhuma integração externa, usa o mesmo motor de orçamento livre, e o custo de incluí-la agora é baixo comparado a abrir uma fase só para isso depois.

Segurança, LGPD e confiabilidade

Dado financeiro não perdoa descuido. As regras aqui valem desde a primeira linha de código, não como pendência para "depois".

10.1 Regras não negociáveis

Autenticação — D-10 decidida

MFA por app autenticador (TOTP) no desktop e biometria no Android (via WebAuthn/passkey) desde o dia 1 — é um sistema de uso pessoal, mas concentra a visão completa da sua vida financeira num único login.

Trilha de auditoria

Toda leitura e escrita relevante (quem, o quê, quando) registrada — útil tanto para segurança quanto para você mesmo entender o histórico de uma decisão financeira.

Credenciais de terceiros

O sistema nunca guarda usuário/senha de banco. Open Finance funciona por OAuth/consentimento — se alguma integração algum dia pedir login bancário direto, é sinal de alerta, não de atalho.

Minimização de dado

Guardar só o necessário para o produto funcionar. Extrato bruto do banco não precisa virar cópia permanente se o dado derivado (saldo, categorização) já resolve.

Criptografia

Tokens de agregador e campos como CPF/CNPJ criptografados em repouso; tudo em trânsito via TLS. Backups também criptografados.

Direito sobre os próprios dados

Mesmo em uso pessoal, construa exportação e exclusão completa de dados como funcionalidade padrão — é boa prática de LGPD e evita ficar refém do próprio sistema.

ANTIPADRÃO A EVITAR

Não decida "implemento a segurança de verdade quando o sistema já estiver funcionando". Autenticação fraca e dados não criptografados são o tipo de dívida técnica que não dá para pagar depois sem reescrever — e aqui o custo de um vazamento não é abstrato, é sua vida financeira exposta.

10.2 Backup e exportação de dados

Formaliza o card "Direito sobre os próprios dados" (10.1) em formatos concretos, para não ficar como boa intenção vaga:

OPERAÇÃO	FORMATO
Exportação	JSON (completo, para backup/migração) e CSV/Excel (para análise fora do sistema)
Importação	OFX e CSV (extratos bancários — já previsto no capítulo 05)
Backup	Criptografado, com rotina de restauração testada — um backup nunca testado não é um backup

Stack técnica e hospedagem

A escolha de React + Laravel API + MySQL é sólida. O ponto de atenção real é a combinação de versões dentro do Hostgator compartilhado.

DECISÃO FECHADA — D-01 / D-02

Laravel 13 foi lançado em março de 2026 e exige **PHP 8.3** como versão mínima — a documentação pública da Hostgator lista suporte só até PHP 8.2 nos planos compartilhados, o que gerava risco de incompatibilidade. Você confirmou PHP 8.3 disponível no seu plano, então o projeto segue com **Laravel 13**. Vale checar, na configuração do domínio da API no cPanel, se PHP 8.3 está de fato selecionado — planos de hospedagem às vezes reverterem a versão padrão em manutenções.

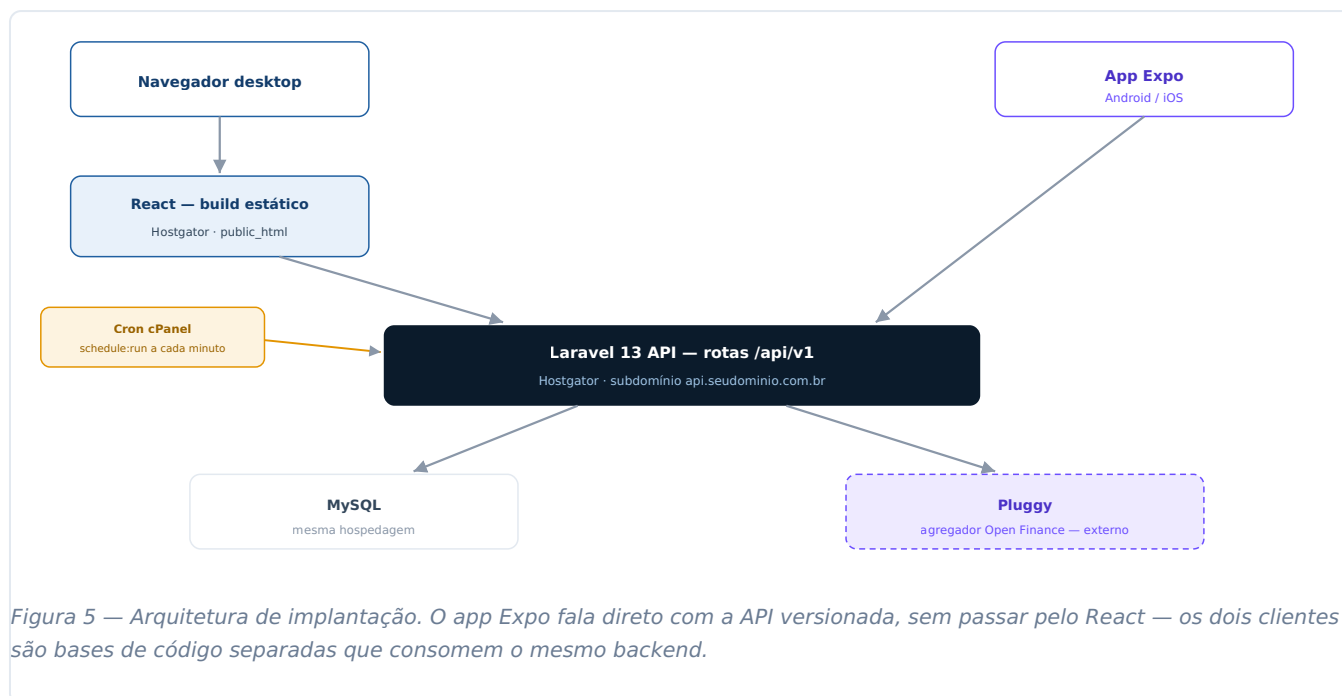


Figura 5 — Arquitetura de implantação. O app Expo fala direto com a API versionada, sem passar pelo React — os dois clientes são bases de código separadas que consomem o mesmo backend.

DECISÃO FECHADA — D-09

O app deixa de ser PWA/TWA e passa a ser um **app Expo** (React Native) nativo, o mesmo caminho que você já usou num projeto anterior — reaproveita experiência real, em vez de aprender TWA do zero para um caso de uso só. Isso muda o princípio 2.3 "uma base de código": agora são **duas** bases de front-end (web e Expo), unidas por uma API versionada (/api/v1/...) e pelas mesmas regras de negócio em **UseCases** — a duplicação fica restrita à camada de apresentação.

11.1 O que funciona bem no Hostgator compartilhado

- **React como build estático** — sem servidor Node necessário; você builda localmente (ou via GitHub Actions) e sobe a pasta `dist/`. Funciona em qualquer hosting que sirva arquivos.

- **Laravel como API pura** — cPanel tem acesso SSH habilitável, Composer e múltiplas versões de PHP por conta; é um cenário comum e documentado.
- **MySQL** — nativo, sem ressalva.
- **Tarefas agendadas** — sem Supervisor (não há acesso root em compartilhado), mas um cron do cPanel chamando `php artisan schedule:run` a cada minuto, combinado com `queue:work --stop-when-empty` agendado no Kernel, cobre bem notificações e sincronizações periódicas.

11.2 O que não funciona — e não force a barra

NÃO TENTE ISSO NO HOSTGATOR COMPARTILHADO

WebSockets/tempo real (Laravel Reverb ou similar) não roda de forma confiável em hospedagem compartilhada — não há processo persistente disponível. Se quiser "atualização instantânea" de saldo, use *polling* (o front pergunta de tempos em tempos) ou *push notification* (nativa no Expo, web push no navegador), não uma conexão persistente.

Node/React rodando no servidor também não é opção — hospedagem compartilhada não roda processos Node de longa duração. O React precisa sempre chegar como build pronto, e o build do Expo (via EAS Build ou local) é feito fora do Hostgator também.

HONESTIDADE SOBRE ESFORÇO

Duas bases de front-end (web e Expo) significam duas vezes o trabalho de tela para toda funcionalidade nova — o simulador, por exemplo, precisa ser construído duas vezes visualmente. O que não duplica é a regra de negócio: como ela mora nos `UseCases` do Laravel (capítulo 11.4), tanto web quanto app chamam a mesma rota versionada e recebem a mesma resposta. Para um projeto pessoal solo, isso é motivo para manter o MVP do app (F3) enxuto — comece só com os módulos de maior uso no dia a dia (dashboard, simulador, captura rápida), não com paridade total desde o primeiro release.

11.3 Casos de uso e services

A escolha de organizar a lógica em casos de uso/services (em vez de "tudo no controller" ou "tudo no model") é acertada para este projeto, porque cada integração externa (agregador bancário, parser de e-mail, scanner de boleto) vira um service substituível sem tocar no resto do sistema — exatamente o padrão que o capítulo 05 recomenda para o `BankAggregatorService``.

11.4 Ponto de partida: laravel-base + padrão DMTA

Você já tem um esqueleto próprio para todo projeto Laravel novo — o repositório `laravel-base` (github.com/deyvidspindola/laravel-base), construído sobre o seu padrão documentado "Padrões Laravel — DMTA". Esse padrão já cobre, quase item a item, o que este capítulo vinha descrevendo de forma genérica — a partir de agora este projeto adota o padrão DMTA como a base literal, não como referência solta.

PADRÃO DMTA	COMO SE ENCAIXA AQUI
PHP 8.3+ / Laravel 13 / MySQL 8	Já é exatamente D-01/D-02 (acima) — confirmação, não mudança
Fila via <code>database + schedule:run</code> , nunca <code>queue:work</code> permanente	Já era o texto de 11.1; o padrão acrescenta três detalhes que faltavam: <code>--stop-when-empty</code> , <code>--max-time</code> abaixo do intervalo do cron, e <code>withoutOverlapping()</code> — sem os três, o esquema funciona por semanas e falha de um jeito difícil de diagnosticar
Clean Architecture — <code>UseCases/Services/Domain/Models</code>	Formaliza o que os capítulos 05, 08 e 11.3 já recomendavam informalmente; passa a ser a estrutura de pastas literal do projeto
Document root fora de <code>public_html</code>	Não estava explícito neste documento — ver alerta abaixo, é o ajuste mais importante desta revisão
Sentry para monitoramento de erro	Novo item de confiabilidade para o capítulo 10 — configurar desde o primeiro bloco de código
Gates/Políticas nativos, sem <code>spatie/laravel-permission</code>	Faz sentido aqui: papéis fixos (dono da conta), sem papéis dinâmicos criados por usuário (capítulo 2.4)
MailerSend (SMTP) para e-mail transacional	Cobre envio (lembretes, alertas). Não cobre o recebimento do capítulo 06 — ver alerta abaixo
Sanctum para token de API (padrão "API pura")	Serve os dois clientes — web e app Expo (D-09) — com o mesmo mecanismo de token, sem sessão de cookie por app

Rotas versionadas desde o início (`/api/v1/...`): quando o app Expo e o React web já consomem a mesma API, uma mudança de contrato sem versão quebra os dois de uma vez. Versionar custa pouco a mais hoje e evita esse problema no dia em que precisar mudar uma resposta.

AJUSTE DE SEGURANÇA QUE ESTE DOCUMENTO AINDA NÃO TINHA

O padrão DMTA exige que a aplicação Laravel fique **fora** de `public_html`, com o domínio (ou o subdomínio da API) apontando só para a pasta `public/` do projeto. Sem isso, `.env`, `storage/` e `vendor/` ficam acessíveis por URL — para um sistema com token de agregador, CPF/CNPJ e credenciais bancárias, esse é o tipo de erro de configuração que anula toda a criptografia do capítulo 10. Teste `api.seudominio.com.br/.env` no navegador logo depois do primeiro deploy — precisa devolver erro 404, nunca o conteúdo do arquivo.

O E-MAIL DE ENTRADA DO CAPÍTULO 06 PRECISA DE PEÇA PRÓPRIA

MailerSend resolve o envio, não o recebimento. O endereço `boletos@seudominio.com.br` (capítulo 06) precisa de um serviço que aceite e-mail de *entrada* e entregue o conteúdo (PDF anexo, texto) para a aplicação — por exemplo, o parse de e-mail de entrada de um provedor como Mailgun ou SendGrid, ou uma caixa IMAP dedicada consultada pelo mesmo cron que já roda `schedule:run`. É uma peça técnica pequena, mas separada do MailerSend — não presuma que ele cobre os dois lados.

Modelo de dados

Visão de alto nível das entidades principais — o suficiente para orientar o desenho técnico detalhado, não o schema final.

ENTIDADE	O QUE GUARDA
Company	Uma empresa do usuário (CNPJ, razão social) — D-03: o usuário pode ter N empresas, não só uma
Context	PF (único, sem <code>company_id</code>) ou PJ (vinculado a uma <code>Company</code>) — chave estrangeira obrigatória em toda tabela financeira, nunca um booleano solto
Account	Conta bancária; origem manual ou agregador
StatementEntry	Lançamento de extrato; valores sempre em centavos, nunca ponto flutuante
Bill	Boleto (a pagar); campo <code>origem</code> (manual / e-mail / scanner / agregador)
Receivable	Conta a receber (salário, cliente, aluguel, Pix esperado); alimenta o fluxo de caixa (capítulo 09)
Category	Categoria de receita/despesa, com <code>parent_id</code> autorreferente — categoria sem pai é categoria-mãe, com pai é subcategoria (D-12). Customizável pelo usuário, vinculada a <code>Bill</code> , <code>Receivable</code> , <code>StatementEntry</code> e <code>CardInvoice</code>
TaxObligation	DARF/DAS; regra de recorrência + status de confirmação
CreditCard / CardInvoice	Limite, dias de fechamento/vencimento, fatura por período
Investment	Posição e aportes por ativo/classe, atualizados manualmente (rentabilidade automática é decisão futura — D-14)
Goal	Meta financeira: valor-alvo, prazo, aportes vinculados, percentual concluído
Simulation	Histórico de simulações rodadas — útil para revisar decisões passadas
AggregatorConnection	Token/consent id do Pluggy, criptografado, com expiração
AuditLog	Quem fez o quê, quando — nunca apagável pelo próprio usuário

OBSERVAÇÕES QUE EVITAM PROBLEMA DEPOIS

Toda entidade financeira carrega `context_id` como obrigatório — isso evita o bug mais perigoso do sistema: um lançamento de uma empresa vazando para a visão PF ou para outra empresa por falta de filtro. Com múltiplas empresas (D-03), o mesmo cuidado vale um nível acima: toda consulta de `Context` precisa checar `company_id` contra o usuário logado, nunca confiar que o `context_id` recebido do front pertence a ele — é a diferença entre uma tela lenta e um vazamento de dado de uma empresa para outra. O campo `origem` em `Bill` / `TaxObligation` / `Account` não é cosmético: é o que sustenta o princípio "nunca finja uma automação que não existe" (capítulo 02).

D-12 decidida: categoria nova nunca abre uma página de cadastro — é sempre um modal de dois campos (nome, categoria-mãe opcional), acionável de qualquer lugar onde se escolhe categoria. Combina direto com o princípio "até 3 cliques para o essencial" (capítulo 2.5): criar uma subcategoria no meio de um lançamento não pode custar mais que isso.

Roadmap em fases

Cinco fases, cada uma entregando valor sozinha. Tamanho relativo, não prazo — você define o ritmo.



Figura 6 — Roadmap em cinco fases. O tamanho do círculo indica o peso relativo de esforço; F4 é cinza porque depende de dado histórico que ainda não existe.

FASE	PORTE	ESCOPO	PRONTO QUANDO
F0	G	Cadastro manual completo — contas a pagar e a receber, boletos, cartões, fiscal, investimentos, categorias — para PF e N empresas; dashboards por contexto e consolidado; login com MFA + biometria; auditoria básica	Você consegue abandonar a planilha atual por um mês inteiro sem sentir falta de nada manual
F1	G	Motor de orçamento livre, simulador de parcelamento (gráficos, CET, cenários, mês mais apertado), semáforo de comprometimento, fluxo de caixa projetado (7/30/90 dias), metas financeiras, motor de e-mail (boletos) e bot do Telegram (captura rápida)	Você usa o simulador e o fluxo de caixa antes de pelo menos uma decisão real de crédito, e um boleto encaminhado por e-mail ou uma despesa mandada por Telegram aparece cadastrado sozinho
F2	M	Integração com Pluggy para contas e extratos; sincronização periódica via cron; reconciliação com lançamentos manuais	Uma conta PF e uma de empresa atualizam saldo sozinhas todo dia, sem intervenção
F3	M	App Expo (Android/iOS) consumindo a API versionada (/api/v1), cartões via agregador, scanner de código de barras pela câmera do app	O app Expo está instalado no seu celular e um boleto físico escaneado aparece cadastrado sozinho
F4	especulativa	Previsão de saldo, recomendações automáticas, alertas inteligentes, explicações sobre variação de gastos	Você tem 6-12 meses de dado limpo acumulado — antes disso, qualquer "inteligência" aqui é só ruído com aparência de precisão

NÃO FAÇA ISSO ANTES DA HORA

F4 é a fase mais fácil de começar cedo demais e a mais cara de fazer mal feita. Um modelo de "previsão de gastos" treinado com dois meses de dado esparsos não prevê nada — só parece que prevê, o que é pior do que não ter a funcionalidade. Trate F4 como ideia registrada, não como próximo passo, até o volume de dado da F0/F1 sustentar.

Decisões em aberto

Catorze decisões mapeadas — treze já fechadas com você. Só uma segue aberta, e ela não bloqueia nada.

ID	FASE	STATUS	PERGUNTA / DECISÃO
D-01	F0	Decidida	Laravel 13 — PHP 8.3 confirmado disponível no plano Hostgator
D-02	F0	Decidida	PHP 8.3 confirmado no plano atual
D-03	F0	Decidida	Multiempresa desde a F0 — usuário pode cadastrar N empresas (CNPJs), não só uma
D-04	F1	Decidida	Simulador com gráficos, CET, cenários comparativos e "mês mais apertado" (capítulo 09)
D-05	F2	Decidida	Pluggy (Meu Pluggy) como agregador de Open Finance
D-06	F1	Decidida	Sem limite de contas/instituições; motor de e-mail e bot do Telegram como portas de captura prioritárias
D-07	F2+	Aberta	Integra Contador tem custo (certificado + consumo por chamada) mas recalcula juros/multa automaticamente — vale revisar depois de rodar F0/F1 (capítulo 07)
D-08	F0	Decidida	Sem consulta automática do BC — cadastro manual/scanner de código de barras é a via definitiva
D-09	F3	Decidida	App nativo via Expo (React Native), consumindo API Laravel versionada (/api/v1)
D-10	F0	Decidida	MFA por app autenticador + biometria no Android, os dois desde o início
D-11	—	Decidida	Uso pessoal — sem intenção de produtizar/vender por ora (capítulo 2.6)
D-12	F0	Decidida	Categorias e subcategorias customizáveis, cadastro rápido via modal (capítulo 12)
D-13	F1	Decidida	Metas financeiras entram na F1, junto com o simulador
D-14	—	Decidida	Sem rentabilidade automática de investimentos por ora — posição e aportes manuais bastam

Detalhe de cada decisão — impacto, opções e histórico — no arquivo `00_REGISTRO_DE_DECIS0ES.md` que acompanha este documento.

Riscos, métricas e próximos passos

O que pode dar errado, como saber se está dando certo, e o que fazer nesta semana — sem esperar nenhuma linha de código.

15.1 Riscos

RISCO	GRAVIDADE	MITIGAÇÃO
Dependência de fornecedor externo (Pluggy) sem fallback	Médio	Isolar em service próprio; manter sempre o caminho manual ativo
Vazamento de credencial de uma empresa expor dado de outra (multiempresa)	Médio	Escopo de autorização por <code>company_id</code> testado explicitamente, nunca só por convenção
Vazamento de dado financeiro	Alto	MFA, criptografia em repouso, trilha de auditoria, nunca logar valores em texto puro
Falha de cron atrasando notificações de vencimento	Médio	Monitorar execução do <code>schedule:run</code> ; alertar se não rodar
Tentativa de scraping do e-CAC para automatizar fiscal	Alto se tentado	Não implementar — manter cadastro manual + regra recorrente (capítulo 07)
Dashboard consolidado incentivando confusão patrimonial	Médio	Consolidado é só leitura; toda transferência PF↔PJ é registrada como tal, nunca implícita
Perseguir rentabilidade automática de investimentos cedo demais	Médio	Manter posição/aportes manuais na F0; tratar cotação de mercado como integração separada (D-14)
Construir "inteligência financeira" (F4) sem dado histórico suficiente	Médio	Gate explícito de 6–12 meses de dado limpo antes de iniciar (capítulo 13)

15.2 Métricas por fase

FASE	MÉTRICA	LINHA DE BASE
F0	Dias seguidos usando o app sem abrir a planilha/apps antigos	Hoje: 0 — tudo espalhado
F1	Nº de decisões de crédito simuladas antes de acontecer	Hoje: 0
F2	% de contas com saldo automático vs. manual	Hoje: 0% automático
F3	App Expo instalado + uso semanal do scanner de boleto	Hoje: não existe

15.3 Próximos passos imediatos

1. Clonar o `laravel-base` como ponto de partida do repositório do projeto, confirmando PHP 8.3 no ambiente Docker local (capítulo 11.4).
2. Criar uma conta gratuita no Meu Pluggy e testar a conexão com um banco seu, para sentir a experiência de consentimento antes de programar qualquer integração.
3. Criar o bot no Telegram via @BotFather — é gratuito e leva minutos; já deixa o token pronto para quando a F1 chegar.
4. Levantar, por 1–2 semanas, todas as contas, cartões, boletos e obrigações atuais (mesmo numa planilha simples), já separadas por categoria e subcategoria — vira o dado semente da F0 e valida se o modelo de dados do capítulo 12 cobre sua realidade.
5. Definir o teto de comprometimento de orçamento que você considera seguro (ex.: 30% da renda livre) — alimenta direto o simulador da F1.